

Re:Learn

Cross-Domain Misconception Graph Tutor for Algebra

Complete build plan, workflow and pitch kit — 24-hour hackathon, 4 people, fully vibe-coded

One-line pitch. Fixing a wrong answer isn't the same as fixing the misconception. Re:Learn finds the exact step where a student's algebra breaks, diagnoses *why*, teaches it visually, and then checks whether the understanding transfers to physics, geometry and code.

Item	Decision
Target users	Students in Classes 7–10 (CBSE/ICSE, ages 12–16) and their teachers
Hero feature	Cross-domain transfer verification — the thing existing tutors don't do
Diagnosis approach	Symbolic-first: SymPy step checking + generate-and-match rules, constrained LLM fallback
Scope	6 algebra misconceptions, each with a visual proof, a retry and a transfer test
Team	4 people, one module each, built against a shared JSON contract
Stack	React + Vite + Tailwind · FastAPI · SymPy · SQLite · one LLM API with vision · React Flow · Recharts

Contents

- 1. Problem and positioning
- 2. Feature set: core, wow layer, roadmap
- 3. Full student workflow (with wow-layer hooks)
- 4. Teacher workflow
- 5. Step-by-step workflow detail
- 6. The 6 misconceptions and transfer tests
- 7. Misconception graph
- 8. System architecture and stack
- 9. Diagnosis engine: generate-and-match
- 10. Shared JSON contract and API
- 11. Team split
- 12. 24-hour timeline
- 13. Evaluation plan
- 14. Demo script
- 15. Judge Q&A; prep
- 16. Risks and team rules

1. Problem and positioning

Most practice apps mark an answer red and show the correct solution. The student copies the fix, gets the next identical question right, and the underlying misconception survives. It reappears later — often in a different subject, where nobody connects it back to algebra.

Research systems and products (Carnegie Learning's Cognitive Tutor, ASSISTments, Eedi) already detect misconceptions. Re:Learn's contribution is narrower and easier to defend:

Existing tutors	Re:Learn
Check the final answer	Find the exact step where equivalence breaks
Label the error (often opaquely)	Diagnose with step-level evidence the student can read
Re-test with a similar algebra question	Re-test in algebra, then in physics / geometry / code
Count wrong answers per class	Show misconception chains, root concepts and transfer failures

Positioning line for judges: "Existing systems detect a misconception. We verify whether the fix actually transfers." Don't claim the product is patent-worthy — say it's a novel combination and back it with a real accuracy number.

2. Feature set

Core — must work perfectly (this is the product)

#	Feature	What it does	Owner
C1	Step-level error localisation	SymPy checks each step against the previous one and highlights the first step that is no longer equivalent.	P1
C2	Generate-and-match diagnosis	Apply each misconception transformation to the previous step; the one that reproduces the student's step is the diagnosis, with exact evidence.	P1
C3	Constrained LLM fallback	When no rule matches, the LLM chooses from the fixed label list or returns 'unknown'. It never invents labels.	P1
C4	Visual proofs	Six pre-built animations (area model, square model, number line, balance scale, tile grouping, sign pattern) with LLM explanation around them.	P3
C5	Same-domain retry	New problem targeting the same misconception. Correct → 'resolved in algebra'. Wrong → alternate explanation (max 2 loops).	P2
C6	Cross-domain transfer test ★	Same concept in physics, geometry or programming. Outcome: 'Transfer verified ✓' or 'Persists in a new context ✗'.	P2 + P4
C7	Live misconception graph	Detected node and its root concept glow; turns green when transfer passes, stays orange when it fails.	P3

Wow layer — lite versions, added only after the core loop works (hour 13+)

#	Feature	Lite implementation	Plugs into	Owner
W1	Photo of handwritten work	Vision model returns the steps as a JSON list; they enter the normal pipeline.	Step 2	P2

#	Feature	Lite implementation	Plugs into	Owner
W2	'Explain your thinking' box	Optional sentence sent with the attempt; used as extra evidence by the LLM.	Step 2 → 5	P2
W3	Confidence signal	Time per step, deletions and edit count → low / medium / high. No webcam, no audio.	Step 2 → 6	P2
W4	Multimodal fusion card	Diagnosis card shows steps, photo, explanation and confidence together.	Step 6	P3
W5	Hindi / Bengali toggle	Re-request the explanation in the chosen language.	Step 7	P2
W6	Knowledge evolution timeline	Per-student chart of misconceptions appearing, resolving and transferring.	Step 10	P3
W7	Teacher dashboard + peer groups	Class heatmap from ~40 seeded students (labelled simulated); list of students sharing a misconception.	Step 10	P4

Wow-layer build priority if time is short: W1 → W7 → W3 → W5 → W6 → W2 → W4.

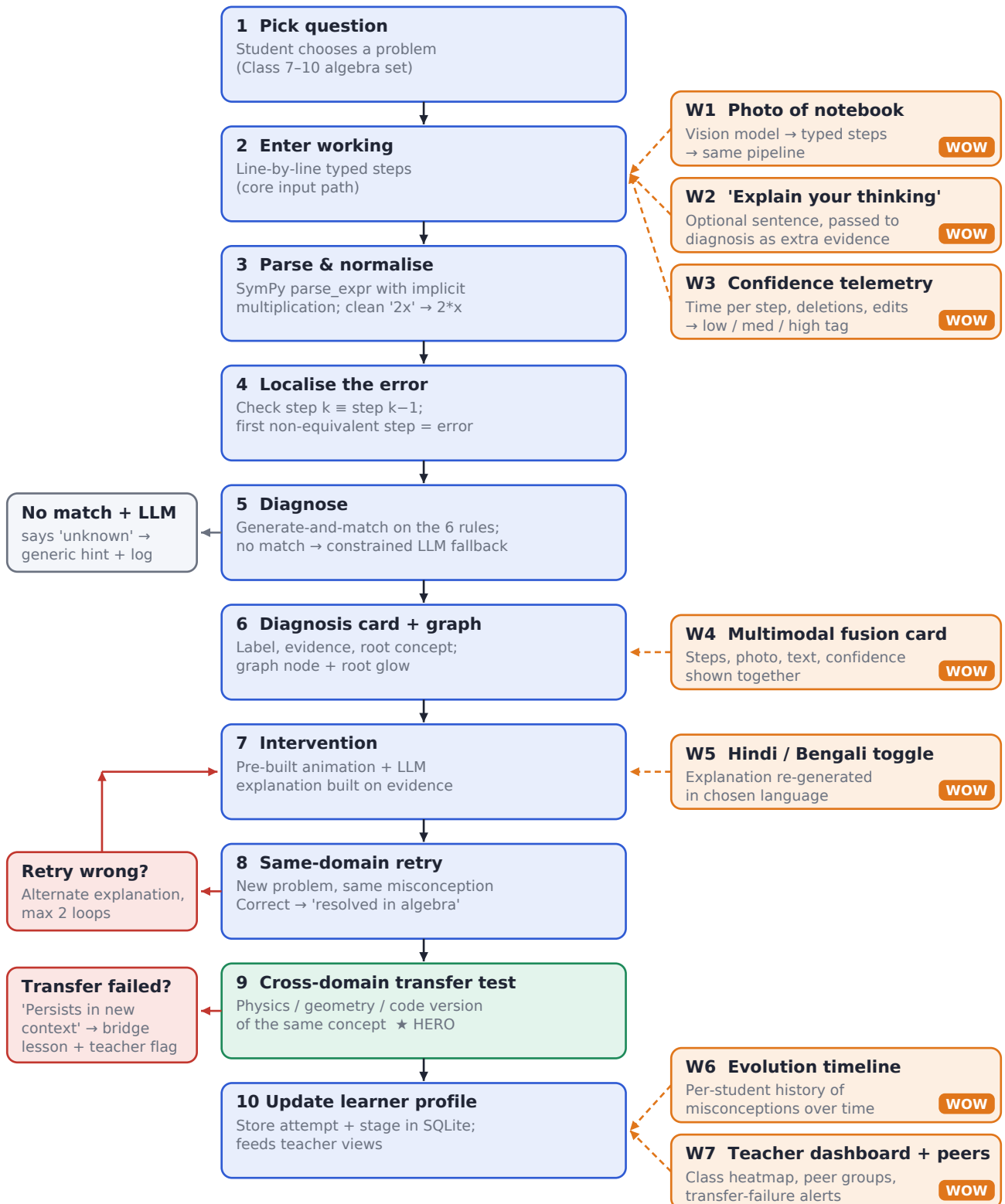
Roadmap — pitch slide only, do not build

- Fine-tuned DeBERTa-v3-small classifier on Eedi/MAP data for free-text student reasoning, compared against a TF-IDF + logistic regression baseline
- Trained resolution model (resolved / persistent / uncertain) once real interaction data exists
- Adaptive curriculum redesign driven by class-level misconception analytics
- Robust handwriting OCR for full notebook pages; more topics (quadratics, slope, fractions)
- Larger misconception graph learned from data rather than hand-built

Deliberately excluded: webcam/microphone emotion detection. Monitoring minors raises consent and privacy problems, facial emotion recognition is scientifically weak, and one judge question can sink the pitch. W3 gives a defensible confidence signal instead.

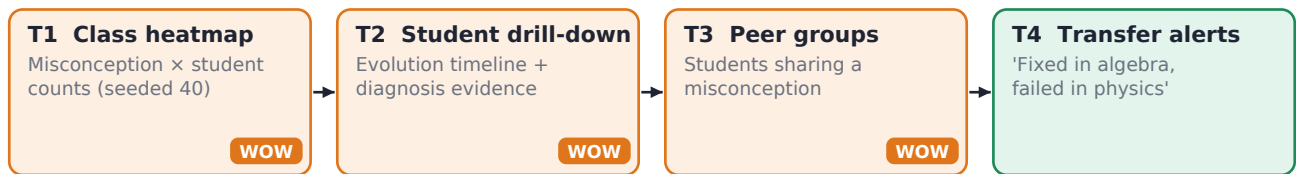
3. Full student workflow

Blue boxes are the core loop. Orange boxes are wow-layer hooks, each pointing at the exact step it plugs into — so the core can ship first and wow features attach later without rewiring anything. Red boxes are failure paths.



 Core step
 Hero feature
 Wow-layer hook
 Failure / loop-back

4. Teacher workflow



Teacher views read only from the learner profile written in step 10 — no extra backend logic needed.

The teacher side never calls the diagnosis engine. It only reads attempts and stages from the learner profile (SQLite), so Person 4 can build it entirely against seeded data from hour 1.

5. Step-by-step workflow detail

Step	Input → Output	Logic	Wow hook
1 Pick question	Student → question_id	Fixed bank: ~4 diagnostic questions per misconception.	—
2 Enter working	Typed lines → steps[]	One input row per line; 'add step' button. Timer runs per row.	W1 photo, W2 explanation, W3 telemetry starts
3 Parse	steps[] → SymPy expressions	parse_expr with implicit multiplication; '=' split into lhs — rhs. Parse failure → ask student to fix that line.	—
4 Localise	expressions → error_step_index	For equations, compare solution sets of step k and k-1 (or simplify(lhs-rhs) ratios). First mismatch is the error step. All equivalent + wrong final → arithmetic slip.	—
5 Diagnose	prev step + error step → label	Run 6 generators on the previous step; match against the student's step. No match → LLM with fixed label list + 'unknown'.	W2 text sent to LLM fallback
6 Diagnosis card	label → card + graph	Show error step, evidence sentence, root concept. Graph node + root glow.	W3 confidence tag, W4 fusion card
7 Intervention	label + evidence → lesson	Play the matching animation; LLM writes 3-4 sentences using the evidence and student's own numbers.	W5 language toggle
8 Retry	new question → pass/fail	Same misconception, new numbers. Fail → alternate explanation, max 2 loops, then move on and flag.	—
9 Transfer ★	transfer question → pass/fail	Hand-verified physics/geometry/code item. Fail → 'persists in new context' + bridge explanation linking the two domains + teacher flag.	—
10 Profile	attempt → SQLite	Store attempt with stage: diagnosed / retry_passed / transfer_passed / transfer_failed.	W6 timeline, W7 dashboard + peers

6. The 6 misconceptions and transfer tests

Person 4 verifies every question and answer by hand. A wrong physics question in front of judges is the worst possible bug.

ID	Wrong step	Root concept	Visual proof	Transfer test
PARTIAL_DISTRIBUTION	$2(x+3) \rightarrow 2x+3$	Distributive law	Rectangle area model split into $2 \cdot x$ and $2 \cdot 3$	Geometry: A rectangle is 2 m wide and $(x+3)$ m long. Write its area.
SQUARE_OF_SUM	$(a+b)^2 \rightarrow a^2+b^2$	Distributive law	Square split into a^2 , b^2 and two ab strips	Geometry: A square garden's side grows from s to $s+2$. How much area is added?
NEGATIVE_DISTRIBUTION	$-(x+5) \rightarrow -x+5$	Integer rules + distribution	Number-line reflection of the whole bracket	Physics: You walk 3 m then 2 m forward; the whole trip is reversed. Displacement?
TRANSPOSITION	$x+5=10 \rightarrow x=10+5$	Equality / inverse ops	Balance scale: remove 5 from both pans	Programming: total = price + tax. Write the line that computes price.
UNLIKE_TERMS	$3x+5 \rightarrow 8x$	Like terms	Grouping tiles: x -tiles vs unit tiles	Physics: Can you add 3 m + 5 s? What does $3x + 5$ mean with units?
NEG_TIMES_NEG	$(-3)(-4) \rightarrow -12$	Integer rules	Pattern: -3×2 , -3×1 , -3×0 , $-3 \times (-1)$... on a number line	Physics: Temperature falls 3°C per hour. Using time = -4 h, how did it compare 4 hours ago?

Expected answers (for the answer key)

ID	Transfer answer
PARTIAL_DISTRIBUTION	$2(x+3) = 2x + 6$
SQUARE_OF_SUM	$(s+2)^2 - s^2 = 4s + 4$ (not 4)
NEGATIVE_DISTRIBUTION	$-(3+2) = -5$ m (not $-3+2 = -1$)
TRANSPOSITION	price = total – tax
UNLIKE_TERMS	No — different units; $3x + 5$ stays as it is
NEG_TIMES_NEG	$(-3)(-4) = +12 \rightarrow$ it was 12°C warmer 4 hours ago

7. Misconception graph

Hand-built for the hackathon: 6 misconception nodes linked to 4 root-concept nodes. Rendered with React Flow.

Root concept	Misconceptions linked to it
DISTRIBUTIVE_LAW	PARTIAL_DISTRIBUTION, SQUARE_OF_SUM, NEGATIVE_DISTRIBUTION
INTEGER_RULES	NEGATIVE_DISTRIBUTION, NEG_TIMES_NEG
EQUALITY_BALANCE	TRANSPOSITION
LIKE_TERMS	UNLIKE_TERMS

Node state	Colour	Meaning
Inactive	Grey	Not seen for this student

Node state	Colour	Meaning
Detected	Orange glow (root glows too)	Diagnosed in step 6
Resolved in algebra	Blue	Retry passed in step 8
Transfer verified	Green	Transfer test passed in step 9
Persists in new context	Red ring	Transfer test failed

Chains show up naturally: a student who fails SQUARE_OF_SUM and later NEGATIVE_DISTRIBUTION lights DISTRIBUTIVE_LAW twice — the graph makes the shared root visible to the teacher.

8. System architecture and stack

```
Student input (typed steps | photo -> vision model | optional explanation | telemetry)
  |
  v
FastAPI /attempt
  |
  v
SymPy step checker -----> error_step_index
  |
  v
Generate-and-match --match--> diagnosis {label, evidence, root_concept, source:"rule"}
  | no match
  v
Constrained LLM fallback --> label from fixed list | "unknown"
  |
  v
/intervention -> animation id + LLM explanation (EN / HI / BN)
  |
  v
/retry -> /transfer -> stage update
  |
  v
SQLite learner profile -> graph, timeline, teacher dashboard, peer groups
```

Layer	Choice	Why
Frontend	React + Vite + Tailwind	Fast to vibe-code, one codebase for student and teacher views
Graph	React Flow	Nodes, edges and colour states with almost no custom code
Animations	SVG + Framer Motion (or D3)	Hand-built, deterministic, look polished
Charts	Recharts	Heatmap and timeline in an hour
Backend	FastAPI (Python)	Same language as SymPy
Maths engine	SymPy	Parsing, equivalence checks, generators
AI	One LLM API with vision (or local Qwen via Ollama for text)	Fallback diagnosis, explanations, photo → steps, translation
Storage	SQLite	Zero setup. No Postgres, Firebase or auth in 24h

9. Diagnosis engine: generate-and-match

The misconception generator does three jobs: it diagnoses student steps, it creates practice variants, and it produces wrong-answer examples for testing. One piece of code, three features.


```

from sympy import simplify, Eq, solve
from sympy.parsing.sympy_parser import (parse_expr, standard_transformations,
                                         implicit_multiplication_application)
T = standard_transformations + (implicit_multiplication_application,)

def parse_eq(s):
    lhs, rhs = s.replace("^", "**").split("=")
    return Eq(parse_expr(lhs, transformations=T), parse_expr(rhs, transformations=T))

def equivalent(a, b, var):
    return set(solve(a, var)) == set(solve(b, var))

def first_error(steps, var):
    eqs = [parse_eq(s) for s in steps]
    for k in range(1, len(eqs)):
        if not equivalent(eqs[k-1], eqs[k], var):
            return k
    return None

GENERATORS = {
    # each returns the WRONG step a student with
    "PARTIAL_DISTRIBUTION": partial_distribute, # that misconception would write
    "SQUARE_OF_SUM": square_each_term,
    "NEGATIVE_DISTRIBUTION": negate_first_term_only,
    "TRANSPOSITION": move_without_inverting,
    "UNLIKE_TERMS": merge_unlike_terms,
    "NEG_TIMES_NEG": wrong_sign_product,
}

def diagnose(prev, student):
    for label, gen in GENERATORS.items():
        for candidate in gen(prev):
            # a generator may return several variants
            if simplify(candidate.lhs - candidate.rhs - (student.lhs - student.rhs)) == 0:
                return {"label": label, "source": "rule", "confidence": 0.95}
    return llm_fallback(prev, student) # fixed labels + "unknown" only

```

LLM fallback prompt rules

- Give the model the previous step, the student's step, the optional explanation and the exact list of 6 labels plus 'unknown'.
- Require JSON only: {label, evidence, confidence}. Reject any label outside the list.
- Cap confidence at 0.7 for LLM-sourced diagnoses and show 'source: AI estimate' on the card — honest and explainable.
- The intervention prompt receives the diagnosis and evidence; it explains, it never re-diagnoses.

10. Shared JSON contract and API

Lock this in hour 1. Paste it into every AI session as 'do not change'. Everyone builds against mock data from minute one.

```
{
  "attempt_id": "uuid",
  "student_id": "s_07",
  "question_id": "PD_02",
  "question": "Solve 2(x+3)=14",
  "steps": ["2x+3=14", "2x=11", "x=5.5"],
  "input_mode": "typed | photo",
  "explanation": "I multiplied the bracket by 2", // W2, optional
  "telemetry": {"ms_per_step": [9100, 4200, 3000], // W3
    "deletions": 6, "edits": 3},
  "error_step_index": 0,
  "diagnosis": {
    "label": "PARTIAL_DISTRIBUTION",
    "source": "rule | llm | unknown",
    "confidence": 0.95,
    "evidence": "2 was multiplied with x but not with 3",
    "root_concept": "DISTRIBUTIVE_LAW"
  },
  "confidence_signal": "low | medium | high",
  "language": "en | hi | bn",
  "stage": "diagnosed | retry_passed | transfer_passed | transfer_failed"
}
```

Endpoint	Method	Purpose	Owner
/questions	GET	Question bank (diagnostic, retry, transfer)	P4 data, P1 route
/attempt	POST	Steps in → error step + diagnosis out	P1
/photo-to-steps	POST	Image → steps[] via vision model (W1)	P1/P2
/intervention	POST	Label + evidence + language → animation id + explanation	P1 + P4 prompt
/retry, /transfer	POST	Check answer, update stage	P1
/student/{id}/history	GET	Timeline + graph states (W6, C7)	P1
/class/summary	GET	Heatmap, peer groups, transfer alerts (W7)	P1 / P4

11. Team split

Person	Role	Owns	Watch out for
P1	Diagnosis engine (backend brain)	SymPy parser and equivalence check, the 6 generators, matcher, LLM fallback, FastAPI routes, SQLite, evaluation script	Messy input ('2x', '^', unicode minus). Use implicit-multiplication parsing from the start.
P2	Student experience	Step input screen, photo upload, error-step highlight, intervention, retry and transfer screens, telemetry, language toggle	Keep every screen working off mock JSON until integration.
P3	Visuals (the wow)	6 animations, live misconception graph, evolution timeline, fusion card	Build the 2 hero animations (area model, square) first.
P4	Content, prompts, dashboard, pitch	All questions and answer keys, intervention prompt, teacher dashboard + seeded data, peer groups, real wrong-answer collection, slides, pitch	Verify every transfer question by hand.

12. 24-hour timeline

Hours	P1 Engine	P2 Student UI	P3 Visuals	P4 Content / Dashboard
0-1	All: lock 6 misconceptions, JSON contract, repo, folder structure, mock data			
1-6	Parser, equivalence check, first 3 generators	Input + result screens on mocks	Area-model + square animations, graph skeleton	All questions, retries, transfer items, answer keys; intervention prompt
6-7	Integration #1 — PARTIAL_DISTRIBUTION end to end			
7-13	Remaining generators, LLM fallback, retry/transfer routes	Retry + transfer flow, loop-backs	Remaining 4 animations, graph states	Seed 40 students, dashboard v1, start collecting real wrong answers
13	CHECKPOINT: core loop demo-ready for all 6 misconceptions			
13-18	/photo-to-steps, history + class endpoints	W1 photo, W3 telemetry, W5 language, W2 box	W6 timeline, W4 fusion card	W7 heatmap, peer groups, transfer alerts; slides draft
18	FEATURE FREEZE — nothing new after this			
18-20	Run evaluation, fix worst failures	Bug fixes	Polish animations	Finish eval set, record numbers
20-23	All: 10+ full demo run-throughs, backup screen recording, final slides			
23-24	All: pitch rehearsal, Q&A drill			

Sleep rotation: two people sleep ~2.5 h each between hours 13 and 18 (P3 + P4 first, then P1 + P2). Don't let the whole team crash at hour 20.

13. Evaluation plan

- **Real data only for the headline number.** Collect 50-100 real wrong solutions from friends, juniors or siblings. Synthetic errors from your own generators make accuracy circular.
- **Metrics:** error-step localisation accuracy; diagnosis accuracy (rule only, and rule + LLM); share of cases returned as 'unknown'.
- **Unseen-question check:** evaluate on questions whose numbers and structure weren't used while writing the generators.
- **Report honestly on the slide:** e.g. 'Tested on 60 real student solutions: X% correct step, Y% correct diagnosis.'

14. Two-minute demo script

Time	Beat	What's on screen / said
0:00-0:10	Hook	"Every wrong algebra answer hides a reason. Most apps just mark it red."
0:10-0:35	Diagnosis	Upload a notebook photo of $2(x+3)=14$ solved wrong (W1). The bad step lights up: '2 was multiplied with x but not 3.' Graph node glows.
0:35-0:55	Teach	Area-model animation plays. Flip explanation to Bengali for two seconds (W5).
0:55-1:05	Retry	Solve $5(x-2)=20$ correctly → 'Resolved in algebra.'
1:05-1:30	Hero moment	Transfer test: rectangle area. Student gets it wrong → 'Persists in a new context.' Pause: "This is what every other system misses. The student memorised the fix but didn't understand it."
1:30-1:45	Teacher view	Heatmap, peer groups, one student's evolution timeline (W6, W7).

Time	Beat	What's on screen / said
1:45-1:55	Proof	"Tested on N real student solutions: X% diagnosed correctly, each with evidence."
1:55-2:00	Close	Roadmap slide: fine-tuned classifier, more topics, curriculum redesign.

15. Judge Q&A prep

Likely question	Answer
Isn't this just GPT checking homework?	No. Diagnosis is symbolic: we find the non-equivalent step and match it to known transformations. The LLM only teaches and handles cases rules can't explain — and those are labelled as AI estimates.
How is this different from Eedi or Khan Academy?	They detect misconceptions. We verify whether the fix transfers to other domains, and show teachers where it doesn't.
How accurate is it?	Quote the real evaluation number and say it comes from real student work, not synthetic data.
Did you train a model?	Not in 24 hours, deliberately. Rules give exact evidence for well-defined errors. A fine-tuned classifier for free-text reasoning is our next step (roadmap).
What about privacy for minors?	No webcam, no audio, minimal data. Confidence comes only from timing and edits.
Does it scale beyond 6 misconceptions?	Each new misconception = one generator + one animation + questions. The graph and pipeline don't change.
Why would a school use it?	Teachers with 40–60 students can't diagnose each one. The heatmap and transfer alerts tell them exactly what to reteach.

16. Risks and team rules

Risk	Mitigation
Integration breaks late	JSON contract locked in hour 1; integration at hour 6–7, not hour 20
AI rewrites working code	Commit to git every time something works; small, scoped prompts
Parser fails on student input	Normalise input early; on parse failure ask the student to fix that line
Wrong content in demo	P4 hand-verifies every question and answer key
LLM slow or down during demo	Cache explanations for demo questions; recorded backup video
Feature creep	Hard freeze at hour 18

Team rules

- Core loop before any wow feature. The checkpoint at hour 13 is non-negotiable.
- Every AI session gets the JSON contract pasted in as 'do not change'.
- One person per module; merge often.
- No new features after hour 18, however easy the AI makes them look.
- Run the full demo at least 10 times before pitching.